Trellix Technical Manual

Author: Issa El Mokadem
Program: 2º DAW – I.E.S. Abdera
Project Type: Full-Stack Web Application (Task Board)

---

1. Project Overview

Trellix is a Kanban-style task management web application. It follows a decoupled architecture with a React SPA frontend, a FastAPI REST backend, a MariaDB database, and Docker-based deployment behind an Nginx reverse proxy.

Layer	Technology	Purpose
Frontend	React 18 + Vite + Tailwind CSS	Single-page UI with drag-and-drop boards
State	Zustand	Client-side state management (optimistic UI)
Backend	Python 3.12 + FastAPI	REST API with JWT authentication and RBAC
ORM	SQLAlchemy 2.0	Database abstraction and migrations
Database	MariaDB 10.11	Persistent storage
Proxy	Nginx	Reverse proxy, static file serving
Containers	Docker + Docker Compose	Development and deployment environment

---

2. System Architecture

2.1 Separation of Concerns

Business logic is distributed across three layers. React components are presentational or minimal coordinators — they do not contain business logic.

Layer	Responsibility	Location
Zustand Store	Board/list/card state, optimistic D&D, API orchestration	`frontend/src/store/boardStore.js`
API Services	HTTP communication with backend	`frontend/src/api/*.js`
FastAPI Endpoints	Authentication, CRUD, validation, authorization	`backend/app/api/v1/endpoints/*.py`
SQLAlchemy Models	Database schema and relationships (anemic domain model)	`backend/app/models/*.py`

Dashboard and Admin pages use local `useState` for simpler state (boards list, user table) where a global store adds unnecessary complexity.

2.2 Data Flow

```
1 React Component → Zustand Store → API Service → FastAPI → SQLAlchemy →
  MariaDB
2        ↑                ↑               ↑
3        └──────── state subscription ────┘
```

Drag-and-drop uses **optimistic UI**: the store updates instantly via `moveCardOptimistic()`, while the backend persists asynchronously via `PATCH /cards/{id}/move`. On failure, state rolls back with `fetchBoard()`.

2.3 Request Flow (Example: Create Card)

1. User clicks "Add Card" in ListCard component
2. `useBoardStore.createCard(listId, title)` dispatches
3. `cardService.create(listId, { title })` → `POST /api/v1/cards/`
4. FastAPI validates input via Pydantic schema
5. `card_service.py` inserts row via SQLAlchemy

6. Response returns to store, which updates normalized `cards` state
7. React re-renders ListCard with new CardItem appended

---

3. Project Structure

```
trellix/
├── docker-compose.yml              # 4 services: nginx, backend, frontend, mariadb
├── nginx/
│   └── nginx.conf                   # Reverse proxy configuration
├── backend/
│   ├── Dockerfile
│   ├── alembic/                     # Database migrations
│   │   ├── env.py
│   │   └── versions/                # 6 migration scripts
│   ├── app/
│   │   ├── main.py                  # FastAPI app entry point
│   │   ├── api/v1/endpoints/
│   │   │   ├── auth.py              # POST /register, /login, /me
│   │   │   ├── users.py             # GET /users, DELETE /users/{id}, block/unblock
│   │   │   ├── boards.py            # CRUD /boards
│   │   │   ├── lists.py             # CRUD /lists, GET /lists/board/{id}
│   │   │   └── cards.py             # CRUD /cards, PATCH /cards/{id}/move
│   │   ├── core/
│   │   │   ├── config.py            # Environment-based settings
│   │   │   ├── security.py          # JWT encode/decode, password hashing
│   │   │   └── exceptions.py        # Custom error handlers
│   │   ├── db/
│   │   │   ├── base.py              # SQLAlchemy declarative base
│   │   │   └── session.py           # Database session factory
│   │   ├── db_init.py               # Programmatic database migrations & seeding
│   │   ├── models/
│   │   │   ├── user.py              # User model
│   │   │   ├── board.py             # Board model
│   │   │   ├── list.py              # List model
│   │   │   └── card.py              # Card model
│   │   ├── schemas/                 # Pydantic request/response models
│   │   └── services/                # Business logic layer
│   └── tests/
│       ├── conftest.py              # Shared fixtures (auth_headers, db_session)
│       ├── test_auth.py
│       ├── test_boards.py
│       ├── test_lists.py
│       ├── test_cards.py
│       ├── test_admin.py
│       ├── test_profile.py
│       └── unit/
│           └── test_user_service.py
├── frontend/
│   ├── Dockerfile
│   ├── vite.config.js
│   ├── tailwind.config.js
│   ├── index.html
│   └── src/
│       ├── main.jsx                 # React entry point
│       ├── App.jsx                  # Router + AuthProvider
```

```
│       ├── context/
│       │   └── AuthContext.jsx       # JWT token, user, login/logout
│       ├── store/
│       │   └── boardStore.js         # Zustand: boards, lists, cards, D&D
│       ├── api/
│       │   ├── axios.js              # Configured Axios instance (baseURL, interceptors)
│       │   ├── boardService.js       # Board CRUD
│       │   ├── listService.js        # List CRUD
│       │   ├── cardService.js        # Card CRUD + move
│       │   ├── userService.js        # User profile
│       │   └── adminService.js       # Admin user management
│       ├── pages/
│       │   ├── Landing.jsx           # Public landing page
│       │   ├── Login.jsx             # Login form
│       │   ├── Register.jsx          # Registration form
│       │   ├── Dashboard.jsx         # Board grid + create modal
│       │   ├── BoardView.jsx         # Kanban board with D&D
│       │   ├── Admin.jsx             # User management table
│       │   └── Profile.jsx           # User settings
│       └── components/
│           ├── board/
│           │   ├── CardItem.jsx      # Draggable task card
│           │   ├── CardModal.jsx     # Card edit modal
│           │   └── ListCard.jsx      # Column container with inline add
│           └── common/
│               ├── Navbar.jsx        # App shell header
│               ├── Sidebar.jsx       # Navigation sidebar
│               ├── Footer.jsx        # App footer
│               ├── WeatherWidget.jsx # Weather sidebar widget (open-meteo)
│               ├── ConfirmModal.jsx  # Reusable confirmation dialog
│               ├── ProtectedRoute.jsx# Auth guard
│               ├── PublicRoute.jsx   # Redirects if authenticated
│               └── DnDErrorBoundary.jsx # D&D error recovery
└── docs/
    ├── TrellixFinalReport.pdf             # Project Final Report
    ├── TrellixIncidentManagement.pdf      # Incident Management Procedure
    ├── architectural_audit.md             # Architecture Audit and Defense Guide
    ├── technical-manual.md                # Technical Reference Manual (This document)
    └── user-manual.md                     # End-user operations guide
```

---

4. Setup & Installation

4.1 Prerequisites

- Docker & Docker Compose
- Git

4.2 Development Setup

```bash
# Clone the repository
git clone https://github.com/issx03/daw-final-proyect-ielm.git
cd daw-final-proyect-ielm

# Start all services
docker compose up -d
```

```
# Verify services are running
docker compose ps
```

This starts four containers: `nginx` (port 80), `backend` (FastAPI), `frontend` (Vite dev server), and `mariadb` (port 3307).

During system startup, database initialization is handled programmatically by `db_init.py` inside the backend container. It polls MariaDB until the port is open and accepting requests (`wait_for_db()`), then executes programmatic Alembic migrations up to the `head` revision (`run_migrations()`), and finally executes database seeding (`init_db()`) to guarantee the default admin user exists before FastAPI begins accepting traffic.

4.3 Environment Variables

Backend configuration is managed in `backend/app/core/config.py` using Pydantic `BaseSettings`. Key variables:

Variable	Default	Purpose
`DATABASE_URL`	`mysql+pymysql://user:pass@mariadb:3306/trellix`	Database connection string
`SECRET_KEY`	*(set in docker-compose)*	JWT signing key
`ACCESS_TOKEN_EXPIRE_MINUTES`	`30`	JWT token lifetime
`CORS_ORIGINS`	`["<http://localhost:5173>"]`	Allowed frontend origins

4.4 Running Tests

```
# All backend tests
docker compose exec -e PYTHONPATH=/app backend pytest tests/ -v

# Single test file
docker compose exec -e PYTHONPATH=/app backend pytest tests/test_auth.py -v

# Frontend tests
docker compose exec frontend npm test
```

---

5. API Reference

5.1 Base URL

```
http://localhost:8000/api/v1
```

5.2 Authentication Endpoints

Method	Path	Auth	Description
POST	`/auth/register`	No	Register new user (email, username, password)
POST	`/auth/login`	No	Login (username or email + password) → returns JWT
GET	`/auth/me`	JWT	Get current user profile
PATCH	`/auth/me`	JWT	Update own profile (non-admin fields only)
DELETE	`/auth/me`	JWT	Delete own account

5.3 Board Endpoints

Method	Path	Auth	Description
GET	`/boards/`	JWT	List all boards
POST	`/boards/`	JWT	Create board (title, description, color)
GET	`/boards/{id}`	JWT	Get board with nested lists and cards
PATCH	`/boards/{id}`	JWT	Update board
DELETE	`/boards/{id}`	JWT	Delete board (cascades to lists and cards)

5.4 List Endpoints

Method	Path	Auth	Description
GET	`/lists/board/{board_id}`	JWT	Get all lists for a board
POST	`/lists/`	JWT	Create list (title, board_id)
PUT	`/lists/{id}`	JWT	Update list
DELETE	`/lists/{id}`	JWT	Delete list (cascades to cards)

5.5 Card Endpoints

Method	Path	Auth	Description
GET	`/cards/list/{list_id}`	JWT	Get all cards for a list
POST	`/cards/`	JWT	Create card (title, description, list_id)
PUT	`/cards/{id}`	JWT	Update card (title, description, priority, labels)
PATCH	`/cards/{id}/move`	JWT	Move card (target list_id + position)
DELETE	`/cards/{id}`	JWT	Delete card

5.6 Admin Endpoints

Method	Path	Auth	Description
GET	`/users/`	Admin	List all users
DELETE	`/users/{id}`	Admin	Delete user
PATCH	`/users/{id}/block`	Admin	Deactivate user (sets is_active=false)

PATCH	`/users/{id}/unblock`	Admin	Reactivate user (sets is_active=true)

5.7 Authentication Flow

1. Client sends `POST /auth/login` with `{ username: "email_or_username", password: "..." }`
2. Backend returns `{ access_token: "eyJ...", token_type: "bearer" }`
3. Frontend stores token in `AuthContext` (memory only)
4. Axios interceptor in `axios.js` attaches `Authorization: Bearer <token>` to every request
5. `get_current_active_user` dependency validates token and injects user into endpoint
6. On 401 response, interceptor clears token and redirects to `/login`

5.8 RBAC (Role-Based Access Control)

- **User**: can manage boards, lists, cards; cannot access admin endpoints
- **Admin**: inherits all User permissions + user management (`GET /users/` , block, unblock, delete)
- Guards implemented via FastAPI dependencies: `get_current_active_user` for auth, `get_current_active_admin` for admin-only routes

---

6. Database

6.1 Entity-Relationship Model

Entity	Table	Key Columns
User	`users`	id, email (unique), username (unique), password_hash, role, is_active, avatar, timestamps
Board	`boards`	id, title, description, color, user_id (FK → users), timestamps
List	`lists`	id, title, board_id (FK → boards), position, timestamps

Card	`cards`	id, title, description, list_id (FK → lists), position (Float), priority, labels (JSON), timestamps

6.2 Relationships

Relationship	Cardinality	FK
User → Board	1:N	`boards.user_id`
Board → List	1:N	`lists.board_id`
List → Card	1:N	`cards.list_id`

6.3 Database Migrations

Migrations are managed with Alembic. Six migrations track the schema evolution:

Migration	Description
`77c6015ffad6`	Initial schema (users, boards, lists, cards)
`3b6dd50c4e8e`	Added role and is_active to users
`98481097aff6`	Added avatar to users
`f0012f7db523`	Added description and color to boards
`72430dbe7aea`	Added priority and labels (JSON) to cards
`18bbaf8a52e7`	Changed card position from Integer to Float (fractional indexing)

```
# Create a new migration
docker compose exec backend alembic revision --autogenerate -m "description"

# Apply pending migrations
docker compose exec backend alembic upgrade head
```

6.4 Design Decisions

- **Float position**: Card positions use Float instead of Integer, enabling fractional indexing. When inserting between positions 2.0 and 3.0, the new card gets 2.5 — no need to rewrite all subsequent positions.

- **JSON labels**: Card labels are stored as a JSON array in the card row, avoiding a separate junction table. Simplifies queries at the cost of SQL-level label filtering.
- **Programmatic Startup Migrations**: The database schema is fully managed through Alembic migrations, which are run programmatically on container startup via `db_init.py`. This ensures consistent schema state across development and production environments, completely eliminating untracked automatic schema creation.

---

7. Frontend Architecture

7.1 Component Tree

```
<App>
└── <AuthProvider>
    └── <Router>
        ├── <Landing />              ← /
        ├── <Login />                ← /login
        ├── <Register />             ← /register
        ├── <ProtectedRoute>
        │   ├── <Dashboard />        ← /dashboard (board grid)
        │   ├── <BoardView />        ← /boards/:id (Kanban)
        │   │   ├── <ListCard />     ← Column (useDroppable)
        │   │   │   └── <CardItem /> ← Draggable card (useSortable)
        │   │   └── <CardModal />    ← Edit overlay
        │   ├── <Admin />            ← /admin (user management)
        │   └── <Profile />          ← /profile
        └── <PublicRoute>
    ├── <Navbar />
    └── <Sidebar />
        └── <WeatherWidget />        ← open-meteo integration
```

7.2 State Management (Zustand)

The `useBoardStore` is the single source of truth for all board-related state:

State	Type	Purpose
`board`	Object	Current board metadata
`lists`	Array	Lists sorted by position
`cards`	`{ listId: [cards] }`	Cards normalized by list ID
`selectedCard`	Object	Card open in edit modal
`loading`	Boolean	Loading indicator
`error`	String	Error message

Key actions:

Action	Type	Flow
`fetchBoard(id)`	Async	GET board with nested lists/cards → normalizes into store
`createCard(listId, title)`	Async	POST card → appends to cards[listId]
`moveCardOptimistic(activeId, overId)`	Sync	Instantly reorders cards array in store
`moveCardBackend(activeId, containerId, index)`	Async	PATCH /cards/{id}/move → on failure calls fetchBoard() for rollback
`updateCard(id, data)`	Async	PUT card → updates card in store

7.3 API Service Layer

Services under `frontend/src/api/` are plain modules that export functions. They use a shared Axios instance (`axios.js`) pre-configured with base URL and JWT interceptor.

Service	File	Methods
Auth	*(handled by AuthContext directly)*	login, register, getMe
Board	`boardService.js`	getAll, create, getById, update, delete
List	`listService.js`	create, getByBoard, update, delete
Card	`cardService.js`	create, getByList, update, move, delete
Admin	`adminService.js`	getUsers, deleteUser, blockUser, unblockUser

7.4 Drag and Drop (dnd-kit)

dnd-kit Layer	Implementation
Context	`<DndContext sensors={...} onDragEnd={...}>` wraps BoardView
Sensors	Pointer sensor (mouse + touch) with 5px activation distance
Collision	`closestCorners` strategy detects which droppable area the card overlaps
Draggable	`useSortable` hook in CardItem provides `transform`, `transition`, `listeners`
Droppable	`useDroppable` hook in ListCard registers the column as a drop target
Overlay	`<DragOverlay>` renders a visual copy of the card during drag
Error Recovery	`<DnDErrorBoundary>` wraps BoardView — catches D&D errors and rolls back state

---

8. Testing

8.1 Backend Testing

Stack: Pytest + TestClient (FastAPI) + SQLite (in-memory)

Global fixtures in `conftest.py`:

- `auth_headers`: registers a user, logs in, returns `{"Authorization": "Bearer <token>"}`
- `db_session`: creates a fresh in-memory SQLite database per test

Test files:

File	Coverage
`test_auth.py`	Registration, login, token validation, email login
`test_boards.py`	Board CRUD, cascade delete
`test_lists.py`	List CRUD, board association

`test_cards.py`	Card CRUD, move endpoint, position ordering
`test_admin.py`	User listing, block/unblock, delete, self-protection
`test_profile.py`	Self-update, admin field protection (SCRUM-94 regression)
`unit/test_user_service.py`	Bcrypt hashing, password verification

Running tests:

```
# All tests inside container
docker compose exec -e PYTHONPATH=/app backend pytest tests/ -v
```

8.2 Frontend Testing

- Component tests: Vitest for CardModal, Navbar, ProtectedRoute, AuthContext, boardStore
- Location: `*.test.jsx` and `*.test.js` files co-located with source

---

9. Design System: "The Digital Curator"

9.1 Concept

Editorial Moderno / Premium UI — dark theme with glassmorphism and subtle ghost borders.

9.2 Typography

Role	Font	Weight	Tracking
Display / Headings	Manrope	ExtraBold (800)	Tight
Body / Labels	Inter	Medium (500)	Normal

9.3 Visual Tokens

Token	Value	Usage
Ghost Border	`rgba(198, 197, 212, 0.15)`	Card borders, dividers
Glass Background	`backdrop-blur` + semi-transparent	Navbar, modals
Container Radius	`rounded-2xl`	Modals, panels

Card Radius	`rounded-lg`	Cards, buttons
Primary Gradient	135° linear	CTAs, primary buttons
Primary Color	`#7C6BEF`	Brand purple

9.4 Tailwind Configuration

The theme extends the default Tailwind config with custom colors and font stacks defined in `frontend/tailwind.config.js`.

---

10. Deployment

10.1 Container Architecture

Container	Image	Port	Purpose
`nginx`	nginx:alpine	80	Reverse proxy, routes `/api` to backend, `/` to frontend
`backend`	python:3.12-slim	8000 (internal)	FastAPI REST API
`frontend`	node:22-alpine (dev) / nginx:alpine (prod)	5173 (internal)	React SPA
`mariadb`	mariadb:10.11	3306 (internal)	Database

10.2 Nginx Configuration

```
server {
    listen 80;
    server_name localhost;

    # Frontend static files
    location / {
        proxy_pass http://frontend:5173;
    }

    # API proxy
    location /api/ {
        proxy_pass http://backend:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

10.3 Production Build

```
# Build frontend for production
```

```
cd frontend && npm run build

# Start all services in production mode
docker compose -f docker-compose.prod.yml up -d
```

---

11. Development Standards

Standard	Rule
Language	All code, comments, commits, and documentation in English
Commits	Conventional Commits: `feat:`, `fix:`, `refactor:`, `test:`, `docs:`, `chore:`
Component files	One component per file. Keep under ~150 lines where practical
Branching	Feature-based branches merged via Pull Requests
Testing	Backend endpoints require an integration test. Frontend tests for critical UI logic
Pull Requests	Include description, testing notes, and reference to Jira issue

---

12. Key Implementation Decisions

Decision	Choice	Rationale
Drag & Drop library	@dnd-kit/core + @dnd-kit/sortable	Hook-based, modular, React 18 optimized. Avoids jQuery-era DOM manipulation
State sync strategy	Optimistic UI (moveCardOptimistic)	Instant visual feedback; backend sync via PATCH; rollback on failure
Card position type	Float	Fractional indexing avoids rewriting all cards on insert

Labels storage	JSON column	Flexible tagging without junction table
Database init	Programmatic Alembic Migrations	Automated schema wait, migration application (`alembic upgrade head`), and seeding (`init_db`) on container startup
Dashboard state	Local useState (not Zustand)	Board list is simple enough; global store adds complexity without benefit
Execution model	Vertical slicing	Auth → Boards/Lists → Cards+DnD. Each layer fully functional before next starts